

CSCI 4160/6963, ECSE 4965/6965

Reinforcement Learning

Homework 7

Overview

In this homework, you will develop the main reinforcement learning algorithms in the case where the Markov decision process (MDP) is unknown and finite. In particular, you will use either SARSA or off-policy SARSA (i.e., Q-learning) to learn policies for Mountain Car and Pendulum. Unlike the previous assignment, you will no longer build an MDP to estimate the simulator; instead, you will directly learn q -values for the state/action pairs. Since both vanilla SARSA and Q-learning work only for finite-state spaces, you will still discretize the state and action spaces as in the previous assignment (so you will still use the MDP states from the last assignment but you will not estimate the transition or the reward function).

Logistics

All coding assignments in this course will be in Python. If you need help with Python, please talk to me ahead of time, so we can discuss the best way to get familiar with it.

For this assignment, you will be using your own computer, which should be sufficient. The code should not require more than 10 minutes to run on any standard laptop.

You are provided with skeleton starting code. Please use the Python libraries in the `requirements.txt` file that's provided on LMS.

Grading

The main challenge in this assignment is to implement SARSA or Q-learning. It is up to you which one you implement – there should not be a significant difference between the two algorithms on the two benchmarks we are considering. The target undiscounted rewards (as averaged over 100 simulator runs) are the same as last time: 90 (**92 for graduate students**) for Mountain Car and -300 (**-250 for graduate students**) for Pendulum. You will lose 5 points for each 1 point of reward below the target in the case of Mountain Car and 5 points for each 10 points of reward below the target in the case of Pendulum.

Make sure your code prints out some indication of progress as well as all required information – we should not have to insert any print statements in order to find out whether your code works!

Finally, please provide answers to each of the questions below:

- 1) Why is exploration more important in Mountain Car than in Pendulum?
- 2) Why does Pendulum require a more fine-grained discretization than Mountain Car?

Hints and Tips

- Keep in mind Mountain Car requires significantly more exploration than Pendulum. You might need a schedule for ϵ in order for learning to succeed.
- During training, you don't need to specify a time limit for Mountain Car. You can run each episode until you reach the goal. During testing, you should terminate each episode after 1000 steps. This is just to simplify exploration.
- In the Pendulum, both training and test episodes should be set to 200 steps.
- You should evaluate your policy every 10 or 100 training episodes. If your averaged reward (over 100 episodes) is above the goal, you can terminate the training.
- It might be helpful to fix the randomization seed so that we can reproduce your results.
- Keep in mind that both SARSA and Q-learning have a slightly different version for the last step in an episode (i.e., the Bellman equation is only the one-step reward). Make sure to handle that case properly (I have provided a boolean variable in the function definitions for that purpose).
- Make sure to experiment with the learning rate α . If your algorithm keeps bouncing back and forth, your learning rate may be too large.

Submission

Please use LMS to submit a zip file containing 1) your **.py** code (in three files, as provided), along with instructions on how to run it, 2) a **.pdf** file containing your answers to the above questions. The deadline is **11:59pm, Thursday, Nov. 6**.